

INFORME DE PRÁCTICA

Semana 4 y 5 — Patrones Creacionales: Singleton, Factory, Builder

Curso: Lenguajes de POO II
Repositorio:

OBJETIVO

Fortalecer el dominio de los patrones creacionales en la Programación Orientada a Objetos mediante implementaciones en **C++** y **Python**, destacando la importancia de la gestión de instancias, la reutilización de código y la escalabilidad en sistemas complejos.

MARCO TEÓRICO

- **Singleton**: asegura una única instancia global para un recurso compartido (ejemplo: configuración de sistema, conexión a base de datos).
- **Factory Method** y **Abstract Factory**: centralizan la creación de objetos, permitiendo independencia de clases concretas.
- **Builder**: simplifica la construcción de objetos complejos dividiéndola en pasos, útil en objetos con múltiples configuraciones (ejemplo: creación de un “Documento PDF” o “Vehículo”).

Los patrones creacionales son esenciales en sistemas grandes, ya que desacoplan la lógica de creación de la lógica de negocio.

DESARROLLO

- I. Implementar **Singleton** en C++ y probar que se crea una única instancia.
- II. Usar **Factory** en Python para simular diferentes medios de transporte.
- III. Crear un **Builder** en Python que arme un “combo de fast food” (hamburguesa, bebida, papas fritas).

Singleton en C++

También llamado: Instancia única

Singleton es un patrón de diseño creacional que garantiza que tan solo exista un objeto de su tipo y proporciona un único punto de acceso a él para cualquier otro código.

- Implementar Singleton en C++ y probar que se crea una única instancia.
-

IMPLEMENTACION EN C++

```
#include <iostream>
using namespace std;

class Config {
private:
    static Config* instance;

    string idioma;

    Config() {
        idioma = "Español";
    }

public:
    static Config* getInstance() {
        if (!instance)
            instance = new Config();

        return instance;
    }

    void setIdioma(string nuevoIdioma) {
        idioma = nuevoIdioma;
    }

    void mostrarIdioma() {
        cout << "Idioma: " << idioma << endl;
    }

    void showMessage() {
        cout << "Configuración global cargada.\n";
    }
};

Config* Config::instance = nullptr;

int main() {

    Config* obj1 = Config::getInstance();
    Config* obj2 = Config::getInstance();

    obj1->showMessage();
}
```

```

// Comparar direcciones
cout << "\nDirección obj1: " << obj1 << endl;
cout << "Dirección obj2: " << obj2 << endl;

// Verificar si son iguales
cout << "\n¿Son iguales?: "
    << (obj1 == obj2) << endl;

// Mostrar valor inicial
cout << "\nValor inicial:\n";
obj1->mostrarIdioma();

// Cambiar desde obj1
obj1->setIdioma("Inglés");

// Ver desde obj2
cout << "\nDespués del cambio:\n";
obj2->mostrarIdioma();

return 0;
}

```

<https://www.onlinegdb.com/edit/qx2BWORvI>

SALIDA ESPERADA

```

Configuración global cargada.

Dirección obj1: 0x5e49a960b920
Dirección obj2: 0x5e49a960b920

¿Son iguales?: 1

Valor inicial:
Idioma: Español

Después del cambio:
Idioma: Inglés

```

Factory Method - Python

También llamado: Método fábrica, Constructor virtual

- Usar Factory en Python para simular diferentes medios de transporte.

IMPLEMENTACION EN PYTHON

```

class Transporte:
    def entregar(self):
        pass

# Transportes concretos
class Camion(Transporte):
    def entregar(self):

```

```

        return "Entrega por carretera"

class Barco(Transporte):
    def entregar(self):
        return "Entrega por mar"

class Avion(Transporte):
    def entregar(self):
        return "Entrega por aire"

class Tren(Transporte):
    def entregar(self):
        return "Entrega por vía férre"

class Moto(Transporte):
    def entregar(self):
        return "Entrega rápida en moto"

class Bicicleta(Transporte):
    def entregar(self):
        return "Entrega ecológica en bicicleta"

class Helicoptero(Transporte):
    def entregar(self):
        return "Entrega aérea especial"

class Drone(Transporte):
    def entregar(self):
        return "Entrega automática con drone"

# Factory
class Factory:

    @staticmethod
    def get_transporte(tipo):

        if tipo == "camion":
            return Camion()

        elif tipo == "barco":
            return Barco()

        elif tipo == "avion":
            return Avion()

        elif tipo == "tren":
            return Tren()

        elif tipo == "moto":
            return Moto()

        elif tipo == "bicicleta":
            return Bicicleta()

        elif tipo == "helicoptero":
            return Helicoptero()

```

```

        elif tipo == "drone":
            return Drone()

        else:
            return None

# Programa principal
print("=== SISTEMA DE TRANSPORTE ===")
print("Opciones:")
print("- camion")
print("- barco")
print("- avion")
print("- tren")
print("- moto")
print("- bicicleta")
print("- helicoptero")
print("- drone")

tipo = input("\nIngrese tipo de transporte: ")

transporte = Factory.get_transporte(tipo)

if transporte:
    print("\n" + transporte.entregar())
else:
    print("\nTipo de transporte no válido")

```

<https://www.onlinegdb.com/edit/AcKL0P58S>

SALIDA ESPERADA

```

=== SISTEMA DE TRANSPORTE ===
Opciones:
- camion
- barco
- avion
- tren
- moto
- bicicleta
- helicoptero
- drone

Ingrese tipo de transporte: avion

Entrega por aire

```

Builder - Python

También llamado: Constructor

Builder es un patrón de diseño creacional que nos permite construir objetos complejos paso a paso. El patrón nos permite producir distintos tipos y representaciones de un objeto empleando el mismo código de construcción.

- Crear un Builder en Python que arme un “combo de fast food” (hamburguesa, bebida, papas fritas).

IMPLEMENTACIÓN EN PYTHON

```
class Combo:

    def __init__(self):
        self.hamburguesa = None
        self.bebida = None
        self.papas = None

    def mostrar_combo(self):
        print("\n=== COMBO FAST FOOD ===")
        print("Hamburguesa:", self.hamburguesa)
        print("Bebida:", self.bebida)
        print("Papas fritas:", self.papas)

class ComboBuilder:

    def __init__(self):
        self.combo = Combo()

    # Agregar hamburguesa
    def add_hamburguesa(self, hamburguesa):
        self.combo.hamburguesa = hamburguesa
        return self

    # Agregar bebida
    def add_bebida(self, bebida):
        self.combo.bebida = bebida
        return self

    # Agregar papas
    def add_papas(self, papas):
        self.combo.papas = papas
        return self

    # Construir combo final
    def build(self):
        return self.combo

# Crear combo usando Builder
combo1 = (
    ComboBuilder()
    .add_hamburguesa("Hamburguesa Doble")
```

```
.add_bebida("Coca Cola")
.add_papas("Papas Grandes")
.build()
)

# Mostrar combo
combo1.mostrar_combo()
```

<https://www.onlinegdb.com/edit/qm3bDTAVm>

SALIDA ESPERADA

```
=== COMBO FAST FOOD ===
Hamburguesa: Hamburguesa Doble
Bebida: Coca Cola
Papas fritas: Papas Grandes
```